

Universidad del Cauca

Primera entrega

Fredy Alejandro Pizo Muñoz
Santiago Cordoba Campana

Facultad de Ingeniería electrónica y telecomunicaciones
Lab. Ingeniería de Software II

Docente
Julio Ariel Hurtado Alegria

Popayán-Cauca
2026

La elaboración de ítems para las pruebas de Estado Saber Pro exige un proceso riguroso de redacción, validación metodológica y evaluación por pares. Para sistematizar este flujo, el presente proyecto desarrolla un Sistema de Banco de Preguntas tipo Saber Pro, el cual gestiona el ciclo de vida completo de las preguntas —desde su creación en formatos de selección múltiple o estudio de caso, hasta su validación automática y asignación de revisores con notificación por correo electrónico—. A nivel técnico, la solución implementa una arquitectura monolítica modular en capas (Presentación, Controlador, Dominio, Servicios y Persistencia en SQLite) e integra patrones arquitectónicos y de diseño clave (Controlador, Entity, Experto, Singleton, Microkernel, Pipeline y Observer), garantizando un sistema extensible, desacoplado y con validación estricta de reglas de negocio. En este documento se detallan la estructura arquitectónica, los patrones aplicados y los prototipos de interfaz que conforman la aplicación.

HISTORIAS DE USUARIO			
ID	COMO	QUIERO	PARA
HU-01	Autor de preguntas	Crear Preguntas de selección múltiple	Guardar en el banco de preguntas
HU-02	Autor de Preguntas	Editar el estado de mis preguntas	Para que se le asigne un revisor
HU-03	Autor de Preguntas	Ver las preguntas que he creado	Modificarlas en un futuro
HU-04	Administrador	Asignar un revisor	Para que las preguntas sean revisadas

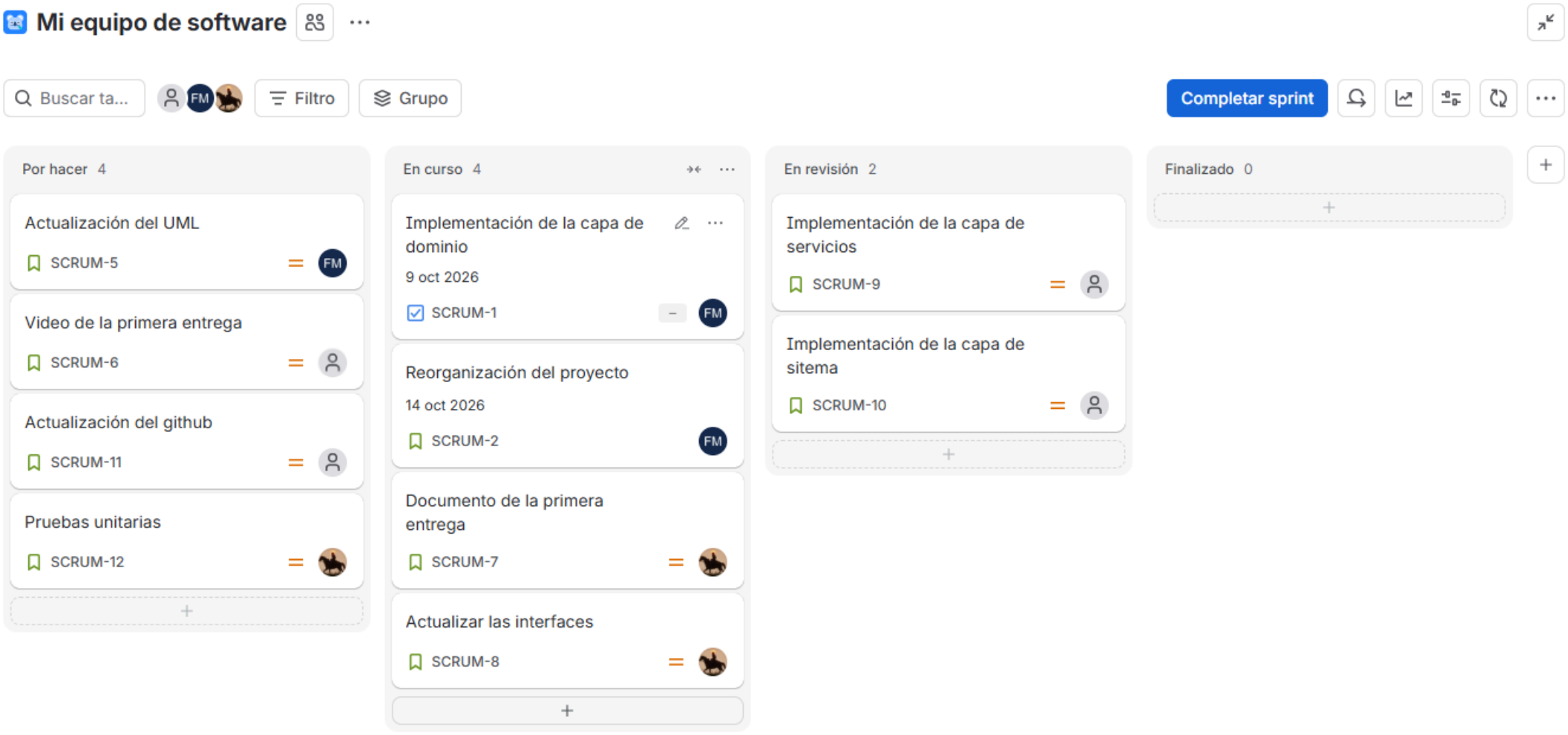
CRITERIOS DE ACEPTACIÓN				
ID HU	CRITERIO	DADO	CUANDO	ENTONCES
HU-01	Validación de campos	Que el usuario está en la interfaz de registro de preguntas	Deja campos vacíos y hace clic en registrar	El sistema muestra un mensaje de error
HU-01	Validación de respuesta correcta	Que el usuario está en la interfaz de registro de preguntas	La respuesta correcta no se encuentra entre las opciones	El sistema muestra un mensaje de error
HU-02	Validación de estado	Que el usuario está en la interfaz de edición de preguntas	Cambia el estado de la pregunta a “Pendiente de revisión”	La pregunta cambia de estado y está disponible para que se le asigne un revisor.
HU-03	Listar preguntas	Que el usuario está en la interfaz de listar preguntas	Ha registrado una pregunta	El sistema mostrará la pregunta creada y podrá editarla
HU-04	Asignación de revisor	Que el usuario está en la interfaz de administrador	Una pregunta se encuentra en estado de “Pendiente de revisión”	El administrador podrá asignar un revisor
HU-04	Notificación de asignación de revisor	Que el usuario está en la interfaz de administrador	Se le asigna un revisor a una pregunta	El sistema notificará al revisor

PROTOTIPOS DE INTERFAZ	
INTERFAZ	DESCRIPCIÓN
Autenticación y Registro de Usuarios	Interfaces de acceso al sistema. uiLogin permite el ingreso de usuarios mediante la validación de sus credenciales (correo/usuario y contraseña), mientras que uiRegisterUser proporciona el formulario para dar de alta nuevas cuentas asignando sus datos personales y rol dentro de la aplicación
Menú Principal y Navegación	Ventana contenedora principal que se despliega tras iniciar sesión. Gestiona la navegación entre los distintos paneles del sistema según el rol del usuario autenticado en la sesión activa y permite cerrar la sesión.
Gestión y Edición de Preguntas	Módulos encargados de la administración del banco de preguntas. pnlMyQuestions lista las preguntas registradas permitiendo su filtrado, paginación y consulta de estado, mientras que el cuadro de diálogo dlgQuestionEditor provee el formulario para crear o editar preguntas (selección múltiple o estudio de caso) y enviarlas a revisión.
Asignación de Revisores	Panel administrativo que permite visualizar las preguntas pendientes de evaluación y asignarles pares evaluadores (revisores), disparando el cambio de estado de la pregunta y la notificación por correo electrónico.

TEST DE EVALUACIÓN DE USABILIDAD				
ID	OBJETIVO	DESCRIPCIÓN	DURACIÓN ESTIMADA	MATERIALES
UT-01 (uTestAuthAndDemo)	Evaluar el flujo de autenticación, registro de usuarios, manejo de sesión y carga de datos de demostración.	Verifica el comportamiento de clsAuthController, clsSession y clsDemoDataSeeder, comprobando el inicio y cierre de sesión, el registro de nuevos usuarios y la inicialización correcta de los datos semilla.	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, clsMemoryRepositories, clsSamples y uTestAuthAndDemo.java.
UT-02 (uTestQuestionController)	Verificar la creación, edición, consulta y gestión de preguntas desde el controlador.	Evalúa las operaciones de clsQuestionController para registrar preguntas, actualizar su información, aplicar criterios de búsqueda y coordinar su transición de estados.	< 1 segundo (Automatizado) / 3 min (Revisión)	Entorno JUnit, clsMemoryRepositories, clsSamples y uTestQuestionController.java.
UT-03 (uTestReviewController)	Comprobar el proceso de asignación de pares evaluadores y la notificación de revisiones.	Prueba la lógica de clsReviewController al vincular revisores a preguntas pendientes (clsReviewAssignment), validando las reglas de asignación y el disparo de notificaciones al observador.	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, clsMemoryRepositories, clsSamples y uTestReviewController.java.
UT-04 (uTestFilterAndPage)	Validar el funcionamiento de los criterios de búsqueda y la paginación de	Verifica que las clases clsSearchCriteria y clsPage filtren correctamente los	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, datos de muestra (clsSamples) y uTestFilterAndPage.java.

	resultados en el dominio.	registros y calculen de forma precisa los subconjuntos de datos, límites y páginas.		
UT-05 (uTestQuestion)	Evaluar la entidad pregunta y las reglas de transición de su ciclo de vida (Patrón State).	Comprueba la integridad de clsQuestion y clsQuestionLifecycle, validando las transiciones permitidas y restringidas entre los estados Borrador (clsStateDraft), Pendiente (clsStatePendingReview) y En Revisión (clsStateInReview).	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, instancias de prueba (clsSamples) y uTestQuestion.java.
UT-06 (uTestMicrokernel)	Verificar el funcionamiento del núcleo extensible y los plugins de tipos de pregunta.	Evalúa clsQuestionMicrokernel e IQuestionPlugin, comprobando la carga y validación específica tanto para preguntas de selección múltiple (clsMultipleChoicePlugin) como de estudio de caso (clsCaseQuestionPlugin).	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, plugins del microkernel y uTestMicrokernel.java.
UT-07 (uTestPipeline)	Evaluar la cadena de validación y normalización de datos (Patrón Pipes and Filters).	Prueba clsPipeline, clsPipelineFactory y los filtros en pkgFilters (longitud mínima, expresiones prohibidas, formato de 4 opciones, respuesta única, coincidencia de contraseñas y normalización).	< 1 segundo (Automatizado) / 3 min (Revisión)	Entorno JUnit, cadenas de entrada en clsSamples y uTestPipeline.java.

UT-08 (uTestServices)	Comprobar los servicios transversales de seguridad, configuración, notificaciones y utilidades.	Verifica el cifrado y validación de contraseñas (clsPasswordHasher), la lectura de parámetros (clsConfig), el notificador de correo (clsEmailNotifier) y el procesamiento de texto (clsText).	< 1 segundo (Automatizado) / 2 min (Revisión)	Entorno JUnit, dobles de prueba de correo/configuración y uTestServices.java.


Mi equipo de software
Buscar ta...
FM
Filtro
Grupo
Completar sprint
Por hacer 4
Actualización del UML
SCRUM-5
FM
Video de la primera entrega
SCRUM-6
Actualización del github
SCRUM-11
Pruebas unitarias
SCRUM-12
En curso 4
Implementación de la capa de dominio
9 oct 2026
SCRUM-1
FM
Reorganización del proyecto
14 oct 2026
SCRUM-2
FM
Documento de la primera entrega
SCRUM-7
Actualizar las interfaces
SCRUM-8
En revisión 2
Implementación de la capa de servicios
SCRUM-9
Implementación de la capa de sitema
SCRUM-10
Finalizado 0

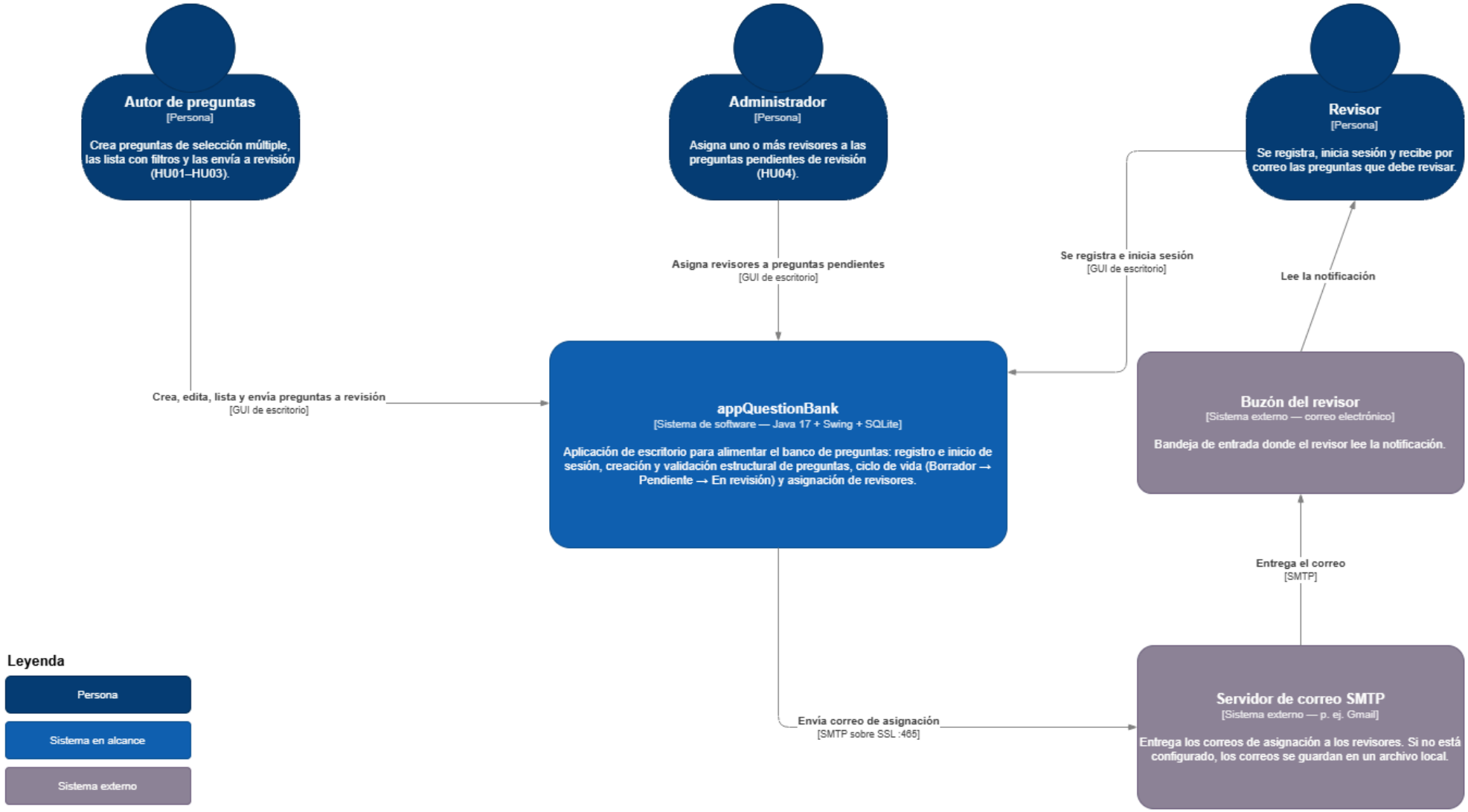
ESCENARIO DE CALIDAD				
CONTEXTO	ESTÍMULO	RESPUESTA	MEDICIÓN DE CALIDAD	RESULTADO ESPERADO
El sistema se encuentra en tiempo de desarrollo y diseño arquitectónico. El código base está estructurado de manera modular (Arquitectura en Capas).	Se solicita agregar un nuevo patrón para la creación de objetos.	El desarrollador encargado extiende el nuevo módulo de creación de objetos con el nuevo patrón.	• **Esfuerzo de desarrollo:** La modificación se completa, prueba e integra en un máximo de 24 horas/hombre por un único desarrollador. • **Impacto en código existente:** El 0% al 3% del código inicial es modificado o reescrito • **Propagación de errores:** Cero fallas o errores en los sistemas que depende de esta instanciación	El sistema seguirá funcionando normalmente sin importar el patrón de creación de objetos.
El sistema se encuentra en tiempo de desarrollo y diseño arquitectónico. El código base está estructurado de manera modular (Arquitectura en Capas).	Se solicita agregar un nuevo filtro o validación para alguna interfaz.	El desarrollador encargado extiende el nuevo módulo con las validaciones o filtros.	• **Esfuerzo de desarrollo:** La modificación se completa, prueba e integra en un máximo de 24 horas/hombre por un único desarrollador. • **Impacto en código**	El sistema incorpora la nueva validación y la usa sin romperse o que suceda algo extraño.

			existente: El 0% al 1% del código inicial es modificado o reescrito ● **Propagación de errores:** Cero fallas o errores en los sistemas que depende de esta u otras validación	
El sistema se encuentra en tiempo de desarrollo y diseño arquitectónico. El código base está estructurado de manera modular (Arquitectura en Capas).	Se solicita agregar una nueva clase(Interfaces, vistas, objetos, etc).	El desarrollador encargado extiende el nuevo módulo con la clase.	● **Esfuerzo de desarrollo:** La modificación se completa, prueba e integra en un máximo de N horas/hombre por un único desarrollador. ● **Impacto en código existente:** El 0% al 10% del código inicial es modificado o reescrito ● **Propagación de errores:** Cero fallas o errores en el sistema.	El sistema no se verá afectado por esta nueva clase por que no depende ni la conoce hasta su despliegue.

El sistema está diseñado bajo un enfoque de Arquitectura Monolítica Modular organizada en capas (Layered Architecture), complementada con principios de Diseño Guiado por el Dominio (DDD - Domain-Driven Design) e Inversión de Dependencias. Es monolítico porque toda la aplicación (interfaz gráfica, lógica de negocio, servicios transversales y acceso a datos) se empaqueta, compila y despliega como una única unidad ejecutable en Java (iniciada desde pkgApp.App). Sin embargo, internamente presenta una alta modularidad y separación de responsabilidades (SoC), dividiendo el sistema en capas lógicas bien definidas que reducen el acoplamiento y facilitan el mantenimiento.

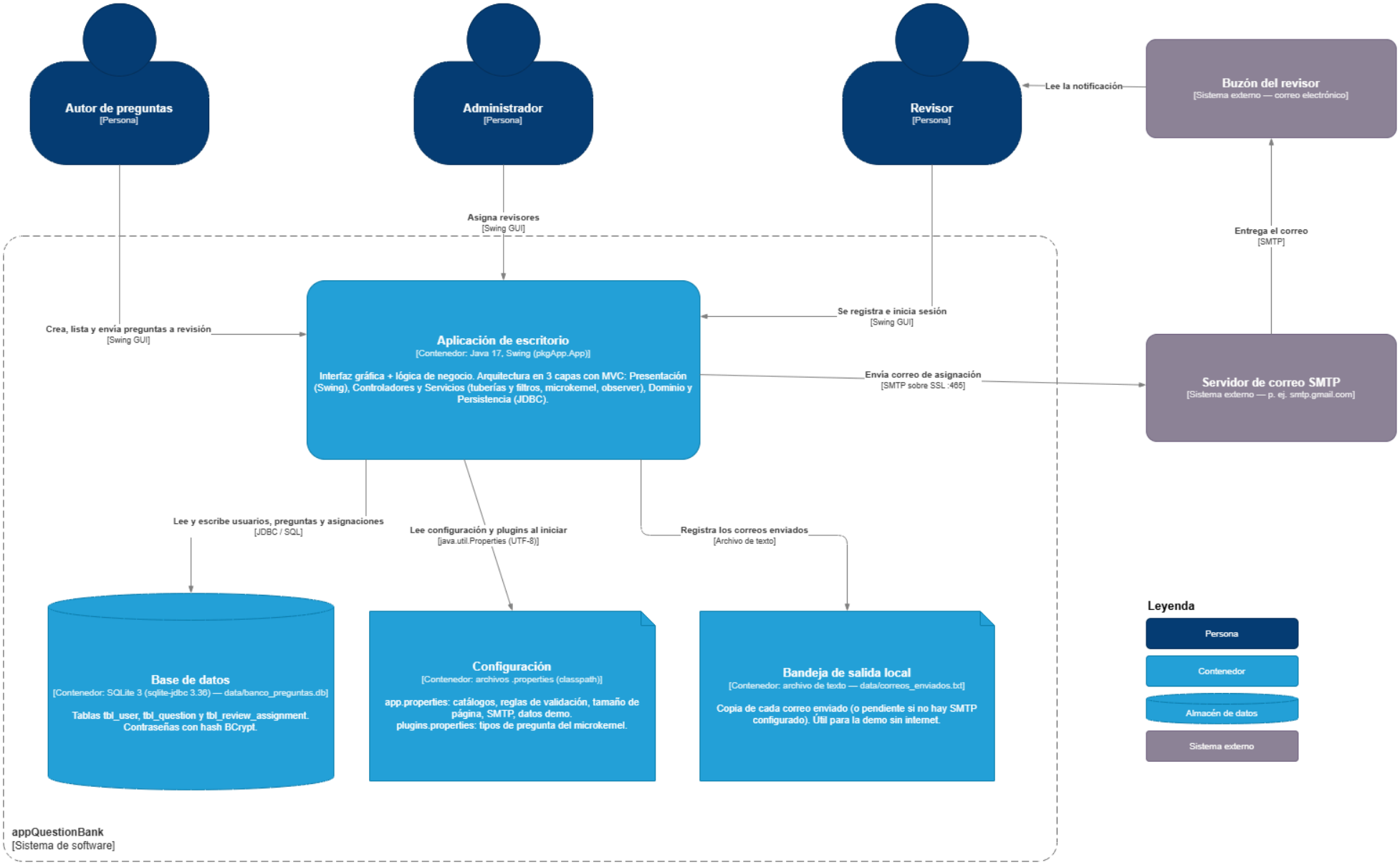
Nivel 1 — Diagrama de contexto del sistema: appQuestionBank

Banco de preguntas Saber Pro (primer corte). Quién usa el sistema y con qué sistemas externos se comunica.



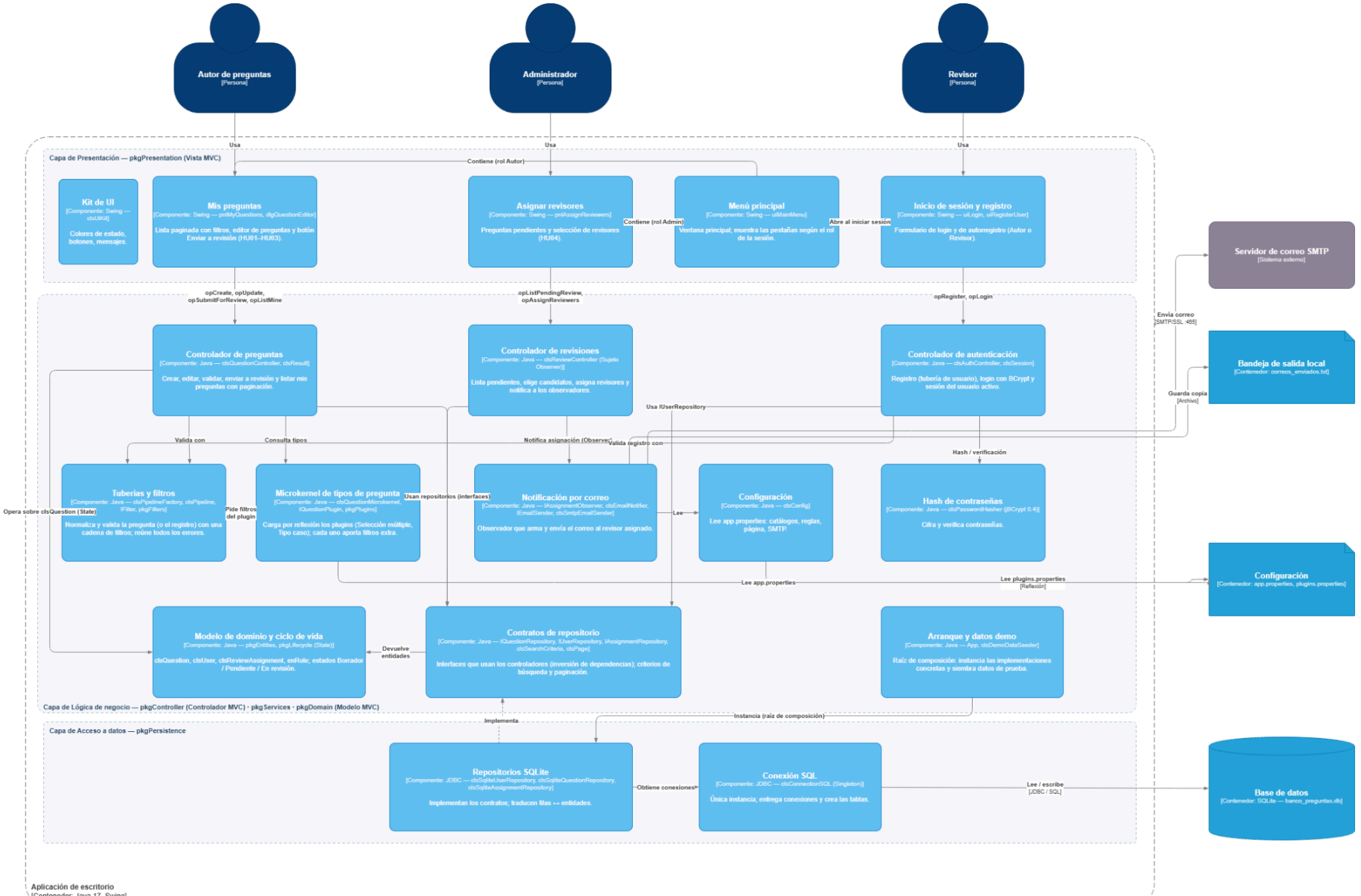
Nivel 2 — Diagrama de contenedores: appQuestionBank

Monolito de escritorio en 3 capas (MVC). Las piezas que se despliegan/ejecutan y dónde se guardan los datos.



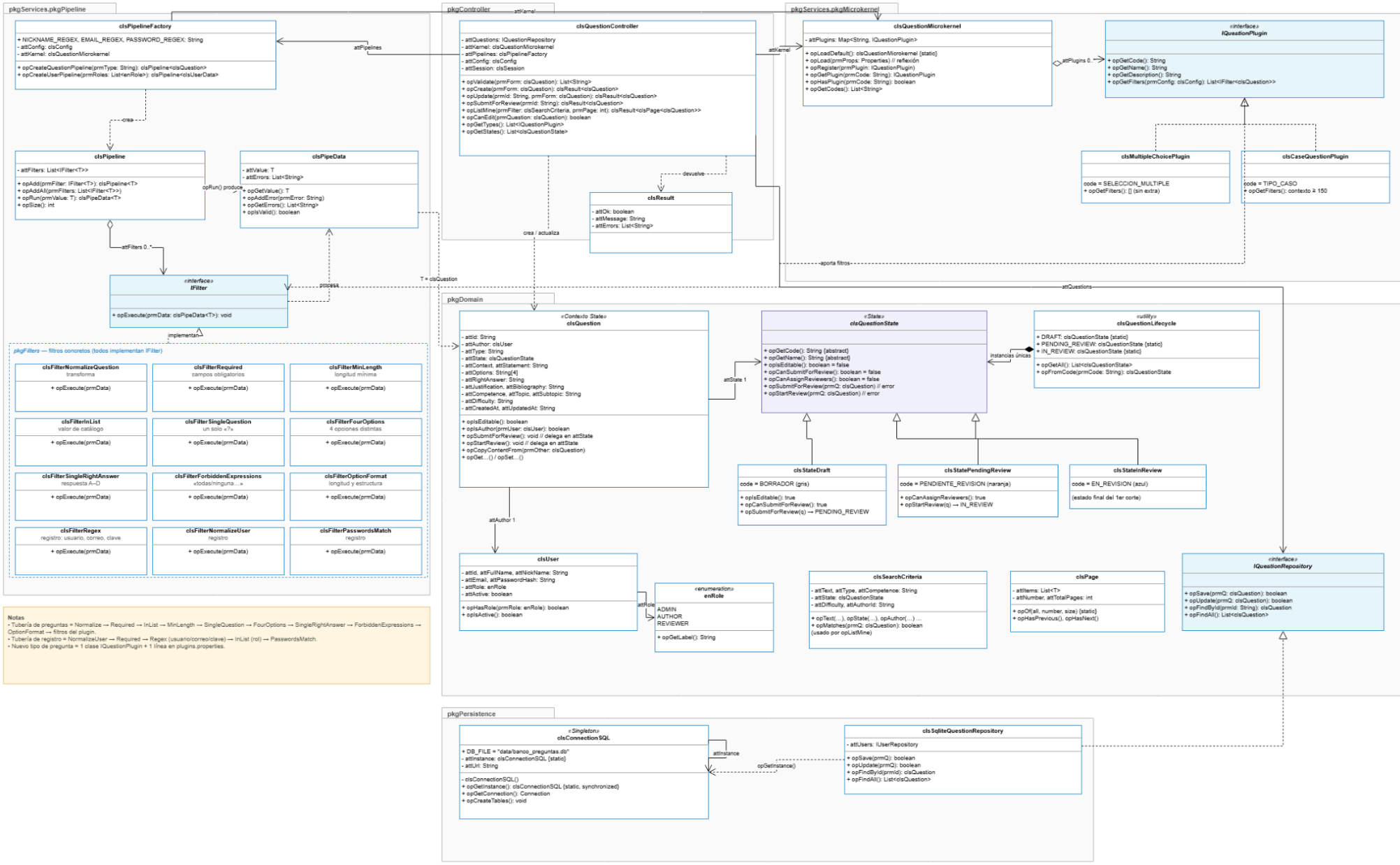
Nivel 3 — Diagrama de componentes: Aplicación de escritorio (Java Swing)

Componentes de cada capa (Presentación → Lógica de negocio → Acceso a datos) y cómo colaboran.



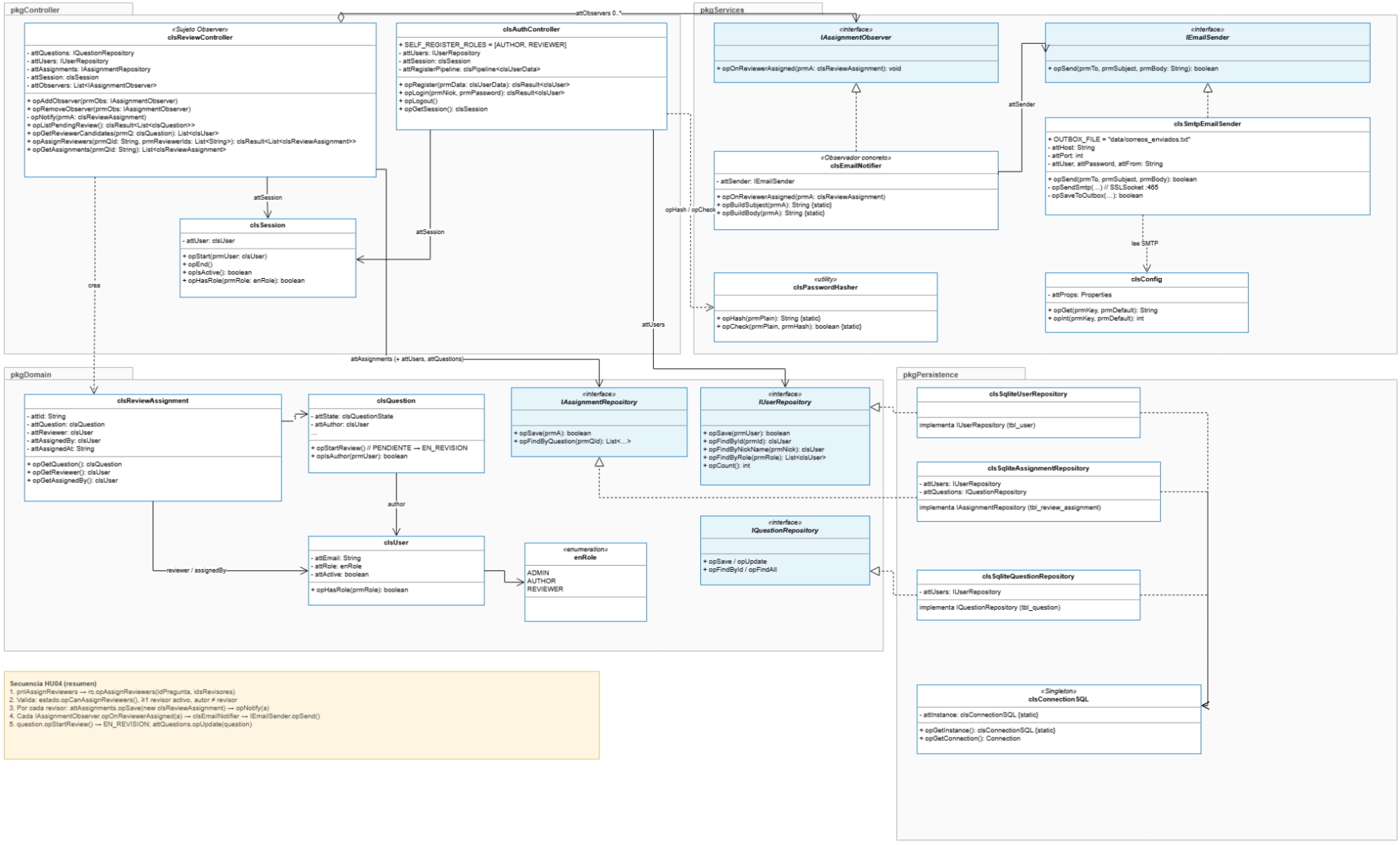
Nivel 4 — Diagrama de código (UML): creación, validación y ciclo de vida de preguntas (HU01–HU03)

Patrones: State (pkgLifecycle), Tuberías y Filtros (pkgPipeline), Microkernel (pkgMicrokernel), Singleton (clsConnectionSQL).



Nivel 4 — Diagrama de código (UML): asignación de revisores y notificación por correo (HU04) + autenticación

Patrones: Observer (clsReviewController sujeto / IAssignmentObserver), inversión de dependencias en repositorios, Singleton.



Secuencia HU04 (resumen)

1. pnlAssignReviewers → rc.opAssignReviewers(idPregunta, idsRevisores)
2. Valida: estado.opCanAssignReviewers(), ≥1 revisor activo, autor ≠ revisor
3. Por cada revisor: attAssignments.opSave(new clsReviewAssignment) → opNotify(a)
4. Cada IAssignmentObserver.opOnReviewerAssigned(a) → clsEmailNotifier → IEmailSender.opSend()
5. question.opStartReview() → EN_REVISION; attQuestions.opUpdate(question)

PATRONES		
PATRÓN	DESCRIPCIÓN	CONTEXTO
Controlador	Recibe y coordina los eventos de la interfaz de usuario, delegando la lógica de negocio a las capas de dominio y servicios.	Paquete pkgController: clsAuthController, clsQuestionController y clsReviewController (junto con clsSession y clsResult), que gestionan las peticiones de las vistas en pkgPresentation.
Entity	Representa objetos del dominio con identidad propia y estado persistente dentro del negocio.	Paquete pkgDomain.pkgEntities: clsQuestion, clsUser, clsUserData, clsReviewAssignment y enRole, persistidos mediante pkgPersistence.
Experto	Asigna la responsabilidad de una operación a la clase que posee la información necesaria para realizarla.	clsQuestion y clsQuestionLifecycle (clsQuestionState y subclases en pkgLifecycle) gestionan el estado y datos de la pregunta; clsUser y clsPasswordHasher validan credenciales.
Singleton	Garantiza la existencia de una única instancia de una clase y ofrece un punto de acceso global a ella.	Aplicado en clsSession (sesión activa del usuario), clsConnectionSQL (conexión única a la base de datos SQLite) y clsConfig (configuración global).
Microkernel	Separa el núcleo funcional del sistema (Kernel) de los módulos de extensión independientes (Plugins).	Paquete pkgServices.pkgMicrokernel: núcleo definido en clsQuestionMicrokernel e IQuestionPlugin; extensiones implementadas en clsCaseQuestionPlugin y clsMultipleChoicePlugin.
Pipeline	Procesa y transforma un flujo de datos a través de una cadena secuencial de filtros independientes.	Paquete pkgServices.pkgPipeline: gestionado por clsPipeline, clsPipelineFactory, clsPipeData e IFilter, aplicando los filtros de validación y normalización del subpaquete pkgFilters.
Observer	Define un mecanismo de suscripción para notificar automáticamente a múltiples objetos sobre eventos o cambios de estado.	Paquete pkgServices.pkgNotification: clsReviewController notifica la asignación de revisores a través de IAssignmentObserver, ejecutando el envío de correos en clsEmailNotifier

		(clsSmtpEmailSender).
State	Cada estado es una clase propia, y es esa clase la que decide qué acciones están permitidas y a qué estado se pasa. La pregunta no usa if ni switch sobre su estado: le delega la decisión al objeto que tenga en ese momento.	Guarda su estado actual en attState, y cuando se llama opSubmitForReview() u opStartReview() lo único que hace es pasarle la llamada a ese estado.